

CSE 331

AUTOMATA AND COMPUTABILITY

Introduction to CFGs

Spring 2026

Today's learning goals

Sipser Ch 1.4, 2.1

- Define context-free grammars
- Test if a specific string can be generated by a given context-free grammar
- Test if a specific string can be generated by a given context-free grammar
- Design a context-free grammar to generate a given language
- Determine if a context-free grammar is ambiguous

The next model of computation

- **Idea:** allow *some* memory of unbounded size
- **How?**
 - Generalization of regular expressions → **Context-free grammars**
 - Generalization for DFA → **Pushdown Automata**

Birds' eye view

All languages over Σ

Context-free languages over Σ

Regular languages over Σ

Finite languages over Σ



Context-free grammar

Sipser Def 2.2, page 102

(V, Σ, R, S)

Variables: finite set of (usually upper case) variables **V**

Terminals: finite set of alphabet symbols **Σ** $V \cap \Sigma = \emptyset$

Rules/Productions: finite set of allowed transformations **R**

$$A \rightarrow u \quad A \in V, u \in (V \cup \Sigma)^*$$

Start variable: origination of each derivation **S**

Context-free language

Sipser p. 104

The **language** generated by a CFG (V, Σ, R, S) is

$\{ w \text{ in } \Sigma^* \mid \text{Starting with the Start variable and applying one or more rules, can derive } w \text{ on RHS} \}$

If $G = (V, \Sigma, R, S)$
the language
generated by G is
denoted $L(G)$.

Notation:
 $S \Rightarrow^* w$

Terminology: sequence of
rule applications is
derivation

An example?

Consider the CFG

$(\{S\}, \{0\}, R, S)$

where R is the following set of rules

$S \rightarrow 0S$

$S \rightarrow 0$

Is this a well-formed definition?

- A. No: there's more than one rule
- B. No: the same LHS gets sent to two different strings.
- C. No: one of the string in the RHS has both variables and literals
- D. Yes.
- E. I don't know.

Context-free language

Sipser p. 104

For CFG $G = (V, \Sigma, R, S)$, $L(G) = \{ w \text{ in } \Sigma^* \mid \text{Starting with the Start variable and applying one or more rules, can derive } w \text{ on RHS} \}$.

What is the language of the CFG $(\{S\}, \{0\}, R, S)$ with $R = \{S \rightarrow 0S, S \rightarrow 0\}$?

- A. $\{0\}$
- B. $\{0, 0S\}$
- C. $\{0, 00, 000, \dots\}$
- D. $\{\epsilon, 0, 00, 000, \dots\}$
- E. I don't know.

Context-free language

Sipser p. 104

What is the language of the CFG ($\{S\}$, $\{0,1\}$, R , S) with R = the set of rules

$$S \rightarrow 0S$$

$$S \rightarrow 1S$$

$$S \rightarrow \varepsilon$$


$$S \rightarrow 0S \mid 1S \mid \varepsilon$$

A. $L(0^*1^*)$

B. $L(0^* \cup 1^*)$

C. $L((0 \cup 1)^*)$

D. $L((0^*1^*)^*)$

E. I don't know.

Designing a CFG

Building a CFG to describe the language
 $\{ abba \}$



Can CFGs
describe simple
sets?

$V = \{ S, T, V, W \}$

$\Sigma = \{ a, b \}$

$R = \{ S \rightarrow aT \quad T \rightarrow bV \quad V \rightarrow bW \quad W \rightarrow a \}$

S

Designing a CFG

Building a CFG to describe the language
 $\{ abba \}$



Can CFGs
describe simple
sets?

$V =$

$\Sigma =$

$R =$

$S =$

What's the **set of terminals** of this CFG?

- A. $\{a, b\}$
- B. $V \cup S \cup \Sigma$
- C. $\{S, a, b\}$
- D. $\{a, b, \epsilon\}$
- E. I don't know.

Designing a CFG

Building a CFG to describe the language
 $\{ abba \}$



Can CFGs
describe simple
sets?

$(\{ S, T, V, W \}, \{ a, b \}, \{ S \rightarrow aT, T \rightarrow bV, V \rightarrow bW, W \rightarrow a \}, S)$

OR

$(\{ S \}, \{ a, b \}, \{ S \rightarrow abba \}, S)$

Designing a CFG

Building a CFG to describe the language

$$\{ a^n b^n \mid n \geq 0 \}$$



We know this
set is not
regular!

Designing a CFG

Building a CFG to describe the language

$$\{ a^n b^n \mid n \geq 0 \}$$

One approach:

- what is shortest string in the language?
- how do we go from shorter strings to longer ones?

Designing a CFG

Building a CFG to describe the language

$$\{ a^n b^n \mid n \geq 0 \}$$

$$V = \{ S \}$$

$$\Sigma = \{ a, b \}$$

$$R =$$

S

Which rules would complete this CFG?

A. $S \rightarrow \epsilon \mid ab$

B. $S \rightarrow \epsilon \mid aS \mid Sb$

C. $S \rightarrow \epsilon \mid aSb$

D. We need another variable other than S.

E. I don't know.

Designing a CFG

Building a CFG to describe the language

$$\{ 0^n 1^m 2^n \mid n, m \geq 0 \}$$



Also not a regular set

Hint: work from the outside in.

Designing a CFG

Building a CFG to describe the language

$$\{ 0^n 1^m 2^n \mid n, m \geq 0 \}$$

Also not a
regular set

Hint: work from the outside in.

$$V = \{ S, T \}$$

$$\Sigma = \{ 0, 1, 2 \}$$

$$R = \{ S \rightarrow 0S2 \mid T \mid \varepsilon, \quad T \rightarrow 1T \mid \varepsilon \}$$

S

Is every regular language a CFL?

- **Approach 1:** start with an arbitrary DFA M , build a CFG that generates $L(M)$.
- **Approach 2:** build CFGs for $\{a\}$, $\{\epsilon\}$, $\{\}$; then show that the class of CFL is closed under the regular operations (union, concatenation, Kleene star).

Approach 1 (Converting regular languages into CFL)

Claim: Given any DFA M , there is a CFG whose language is $L(M)$.

Construction:

Trace computation using variables to denote state

Given $M = (Q, \Sigma, \delta, q_0, F)$ a DFA, define the CFG

$$\mathbf{V} = \{ S_i \mid q_i \text{ is in } Q \}$$

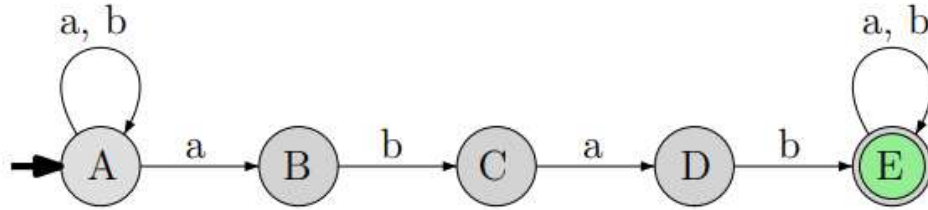
Σ

$$\mathbf{R} = \{ S_i \rightarrow aS_j \mid \delta(q_i, a) = q_j \} \cup \{ S_i \rightarrow \varepsilon \mid q_i \text{ is in } F \}$$

$$\mathbf{S} = S_0$$

Then prove correctness...

Approach 1 (Converting regular languages into CFL)



$$G = \left(\{A, B, C, D, E\}, \{a, b\}, \left\{ \begin{array}{l} A \rightarrow aA, A \rightarrow bA, A \rightarrow aB, \\ B \rightarrow bC, \\ C \rightarrow aD, \\ D \rightarrow bE, \\ E \rightarrow aE, E \rightarrow bE, E \rightarrow \epsilon \end{array} \right\}, A \right)$$

Approach 1 (Converting regular languages into CFL)

Building a CFG to describe the language

$\{w \mid w \text{ start and end with different symbols}\}$

Hint: write the regular expression first then convert the regular expression to CFG

Regular expression: $a(a+b)^*b + b(a+b)^*a$

Approach 1 (Converting regular languages into CFL)

Building a CFG to describe the language

$\{w \mid w \text{ start and end with different symbols}\}$

$$V = \{ S, A, B \}$$

$$\Sigma = \{ a, b \}$$

$$R = \{ S \rightarrow aPb \mid bPa, P \rightarrow aP \mid bP \mid \epsilon \}$$

S

Approach 2

- Build CFGs for $\{a\}$, $\{\varepsilon\}$, $\{\}$; then show that the class of CFL is closed under the regular operations (union, concatenation, Kleene star).

Approach 2

If $G_1 = (V_1, \Sigma, R_1, S_1)$ and $G_2 = (V_2, \Sigma, R_2, S_2)$ are CFGs and G_1 describes L_1 , G_2 describes L_2 , how can we combine the grammars so we describe $L_1 \cup L_2$?

$$V = V_1 \cup V_2 \cup \{S\}$$

$$\Sigma$$

$$R = R_1 \cup R_2 \cup \{S \rightarrow S_1 \mid S_2\}$$

$$S$$

Designing a CFG

Building a CFG to describe the language

$$\{ ww^R \mid w \text{ in } \Sigma^* \}$$

$$V = \{ S \}$$

$$\Sigma = \{ a, b \}$$

$$R =$$

S

Which rules would complete this CFG?

A. $S \rightarrow \epsilon \mid ab$

B. $S \rightarrow \epsilon \mid aSa \mid bSb$

C. $S \rightarrow \epsilon \mid aSb$

D. We need another variable other than S.

E. I don't know.

Designing a CFG – challenge!

Building a CFG to describe the language

$$\{ ww \mid w \in \Sigma^* \}$$

$$V = \{ S \}$$

$$\Sigma = \{ a, b \}$$

$$R =$$

S

Which rules would complete this CFG?

- A. $S \rightarrow \epsilon \mid aS \mid bS \mid SS$
- B. $S \rightarrow \epsilon \mid aSb \mid bSa$
- C. $S \rightarrow \epsilon \mid aSa \mid bSb$
- D. None of these work
- E. I don't know.

Designing a CFG – challenge!

Building a CFG to describe the language

$$\{ ww \mid w \in \Sigma^* \}$$

$$V = \{ S \}$$

$$\Sigma = \{ a, b \}$$

$$R = ?$$

S

This language is not context free!

This can be proved using a
“pumping lemma” for context free languages

Closure properties of CFLs

$G_1 = (V_1, T, P_1, S_1)$ and $G_2 = (V_2, T, P_2, S_2)$

Assumption: $V_1 \cap V_2 = \emptyset$, that is, non-terminals are not shared

Theorem

CFLs are closed under union. L_1, L_2 CFLs implies $L_1 \cup L_2$ is a CFL.

Theorem

CFLs are closed under concatenation. L_1, L_2 CFLs implies $L_1 \bullet L_2$ is a CFL.

Theorem

CFLs are closed under Kleene star.

If L is a CFL $\implies L^*$ is a CFL.

More bad news: CFL not closed under intersection

Theorem

CFLs are *not* closed under intersection.

CFLs are *not* closed under complementation as well.

CFGs in the wild

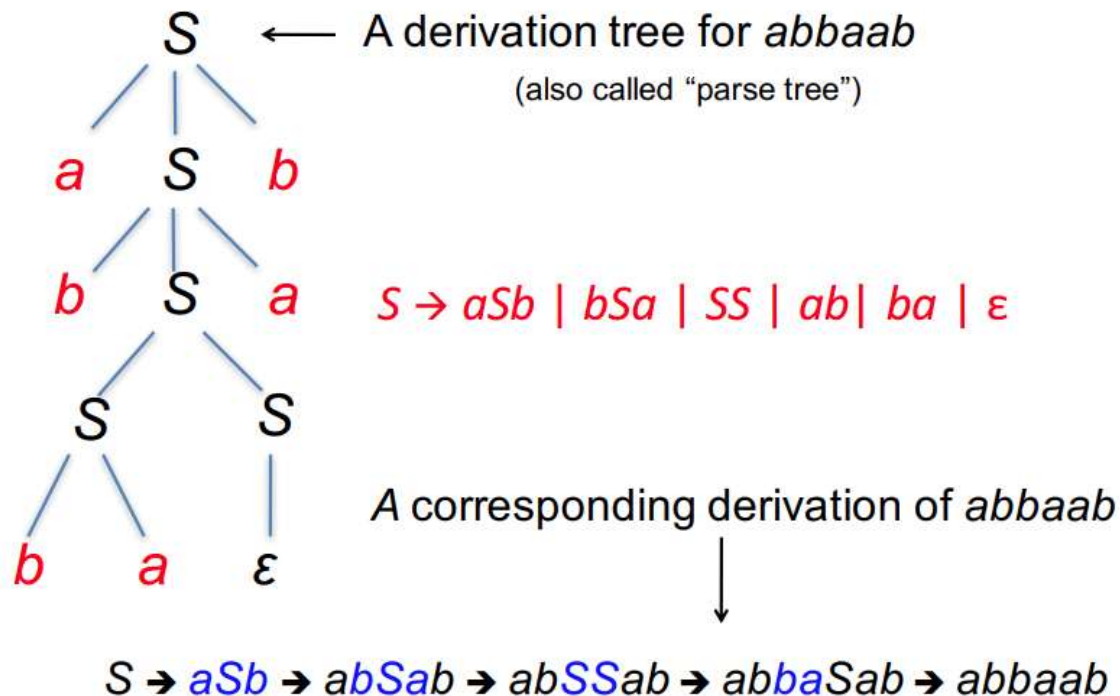
$V = \{E\}$, $\Sigma = \{1, +, x, (,)\}$, $R = \{ E \rightarrow E+E \mid ExE \mid (E) \mid 1 \}$, $S=E$

Describing well-formed arithmetic expressions

Which of the following strings is generated by this CFG?

- A. E
- B. 11
- C. 1+1x1
- D. ϵ
- E. I don't know.

Derivations and parsing

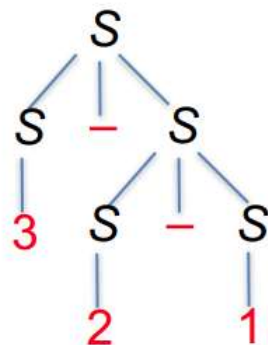


Ambiguity in CFLs

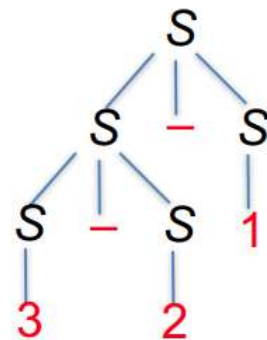
Definition

A CFG G is **ambiguous** if there is a string $w \in L(G)$ with two different parse trees. If there is no such string then G is **unambiguous**.

Example: $S \rightarrow S - S \mid 1 \mid 2 \mid 3$



3-(2-1)



(3-2)-1

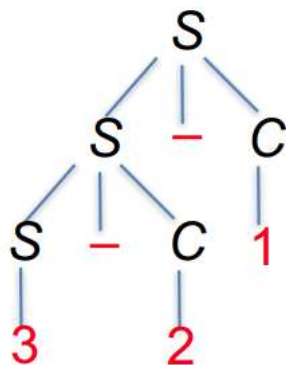
Ambiguity in CFLs

► Original grammar: $S \rightarrow S - S \mid 1 \mid 2 \mid 3$

► Unambiguous grammar:

$S \rightarrow S - C \mid 1 \mid 2 \mid 3$

$C \rightarrow 1 \mid 2 \mid 3$



The grammar forces a parse corresponding to left-to-right evaluation.

$(3-2)-1$

Derivations and parsing

$$E \rightarrow E+E \mid E \times E \mid (E) \mid 1$$

Lots of derivations for $1+1 \times 1$

$$E \Rightarrow E + E \Rightarrow E + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + 1 \times E \Rightarrow 1 + 1 \times 1$$

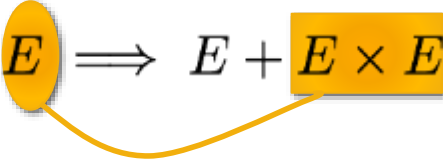
$$E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + 1 \times E \Rightarrow 1 + 1 \times 1$$

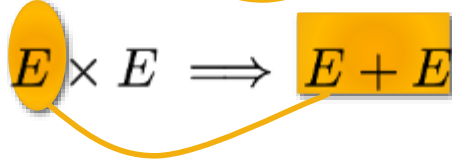
$$E \Rightarrow E + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + 1 \times E \Rightarrow 1 + 1 \times 1$$

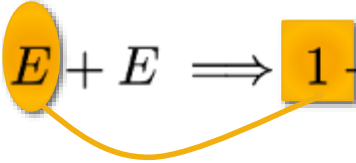
Derivations and parsing

$$E \rightarrow E+E \mid E \times E \mid (E) \mid 1$$

Lots of derivations for 1+1x1

$$E \Rightarrow E + E \Rightarrow E + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + 1 \times E \Rightarrow 1 + 1 \times 1$$


$$E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + 1 \times E \Rightarrow 1 + 1 \times 1$$


$$E \Rightarrow E + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + 1 \times E \Rightarrow 1 + 1 \times 1$$


Derivations and parsing

$$E \rightarrow E + E \mid E \times E \mid (E) \mid 1$$

Lots of derivations

$$E \Rightarrow E + E$$

$$E \Rightarrow E \times E$$

$$E \Rightarrow E + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + 1 \times E \Rightarrow 1 + 1 \times 1$$

A string is **ambiguously derived** in a CFG if it has more than one leftmost derivation i.e. more than one parsing tree

$$1 + 1 \times 1$$

Derivation, Parse Tree and Ambiguity

Consider the following context-free grammar.

$$S \rightarrow PQ$$

$$P \rightarrow aPa \mid aPb \mid aPc \mid cPa \mid bPc \mid a \mid b \mid c$$

$$Q \rightarrow aQ \mid bQ \mid \varepsilon$$

- (a) **Give** a leftmost derivation for the string 'acaacababb'. (3 points)
- (b) **Draw** the parse tree corresponding to the derivation you gave in (a). (2 points)
- (c) **Show** that the given grammar is ambiguous by drawing two more parse trees (apart from the one that you have drawn in (b)) for the given string in (a). (4 points)
- (d) **Write** two strings of length four that have exactly one parse tree in the given grammar. (1 point)

Recap: Context-free languages

Context-free grammar

$G = (V \text{ finite set of variables}, \Sigma \text{ finite set of terminals}, R \text{ finite set of rules}, S \text{ start variable})$

One-step derivation

$uAv \Longrightarrow u w v$ where $u, v, w \in (\Sigma \cup V)^*$ $A \rightarrow w \in R$

Derivation

$u \xRightarrow{*} v$ $u = v$ or $u \Longrightarrow u_1 \Longrightarrow \cdots \Longrightarrow u_k \Longrightarrow v$

Language generated by grammar

$L(G) = \{w \in \Sigma^* \mid S \xRightarrow{*} w\}$